

OTP, DAY 1, BACKEND: the simple step-by-step (copy, paste, run, test in Swagger)

This is the same OTP backend as `DAY1-OTP-BACKEND.pdf`, but as a plain checklist. Do the steps **in order**. Each step says exactly what to do and what you should see. You do **not** need Twilio for this: in "dev mode" the code is printed in your backend console and also returned in Swagger.

The idea in 6 lines

1. A person registers with a mobile number.
2. The server marks the account **"waiting for verification"** (one row in a new table `phone_verifications`).
3. The person asks for a code: the server makes a random 6-digit number and "sends" it (dev mode: prints it in the console).
4. The person types the code back; the server checks it.
5. If it matches, the account is **verified** and login works. Until then, login answers **403 "Please verify your mobile number to continue"**.
6. The code lives 5 minutes, allows 5 wrong tries, and the server stores only a scrambled (hashed) copy, never the code itself.

Old accounts (and your seeded admin) have no row in that table, so they count as verified and are never locked out.

Before you start

- You already applied **CHANGES-7** (it provides `RateLimiter` and the Swagger page). Check that this file exists: `springapp/src/main/java/com/examly/springapp/config/RateLimiter.java`.
- All new Java files go under `springapp/src/main/java/com/examly/springapp/` in the folders named below. The folder `sms` is **new**: create it next to `service`, `model`, etc.
- Package names must match the folder. Every file below already starts with the right `package` line; paste it whole.

Step 1: create 9 new files (just copy each one whole)

Nothing existing is touched in this step. Order does not matter.

Create `springapp/src/main/java/com/examly/springapp/model/PhoneVerification.java`

```
package com.examly.springapp.model;

import jakarta.persistence.*;

/**
 * Phone verification state of one account (customer or instructor). A row with verified = false means
 * "this account must verify its mobile number before it can log in". Accounts without a row (all accounts
```

```

* created before OTP existed, admins, Google logins) count as verified. The real code is never stored,
* only a salted hash of it.
*/
@Entity
@Table(name = "phone_verifications")
public class PhoneVerification {
    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    @Column(nullable = false, unique = true)
    private Long userId;

    private boolean verified;

    @Column(length = 64)
    private String codeHash;

    @Column(length = 32)
    private String salt;

    /** When the current code stops working (milliseconds since 1970). */
    private long expiresAtMs;

    /** Wrong tries on the current code. */
    private int attempts;

    private long lastSentAtMs;
    private long verifiedAtMs;

    public PhoneVerification() {}

    public PhoneVerification(Long userId, boolean verified) {
        this.userId = userId;
        this.verified = verified;
    }

    public Long getId() { return id; }
    public Long getUserId() { return userId; }
    public boolean isVerified() { return verified; }
    public void setVerified(boolean verified) { this.verified = verified; }
    public String getCodeHash() { return codeHash; }
    public void setCodeHash(String codeHash) { this.codeHash = codeHash; }
    public String getSalt() { return salt; }
    public void setSalt(String salt) { this.salt = salt; }
    public long getExpiresAtMs() { return expiresAtMs; }
    public void setExpiresAtMs(long expiresAtMs) { this.expiresAtMs = expiresAtMs; }
    public int getAttempts() { return attempts; }
    public void setAttempts(int attempts) { this.attempts = attempts; }
    public long getLastSentAtMs() { return lastSentAtMs; }
    public void setLastSentAtMs(long lastSentAtMs) { this.lastSentAtMs = lastSentAtMs; }
    public long getVerifiedAtMs() { return verifiedAtMs; }
    public void setVerifiedAtMs(long verifiedAtMs) { this.verifiedAtMs = verifiedAtMs; }
}

```

Create `springapp/src/main/java/com/examly/springapp/repository/PhoneVerificationRepo.java`

```

package com.examly.springapp.repository;

import com.examly.springapp.model.PhoneVerification;
import java.util.Optional;
import org.springframework.data.jpa.repository.JpaRepository;
import org.springframework.stereotype.Repository;

/** Data access for phone verification rows. */
@Repository
public interface PhoneVerificationRepo extends JpaRepository<PhoneVerification, Long> {

```

```
Optional<PhoneVerification> findByUserId(Long userId);  
}
```

Create

springapp/src/main/java/com/examly/springapp/exception/PhoneNotVerifiedException.java

```
package com.examly.springapp.exception;  
  
import org.springframework.http.HttpStatus;  
  
/**  
 * Raised at login when the password is right but the mobile number has not been verified with an OTP yet  
 * (answered with 403). The message is fixed and safe to show; the frontend looks for it to open the  
 * "Verify your number" page.  
 */  
public class PhoneNotVerifiedException extends AppException {  
    public static final String MESSAGE = "Please verify your mobile number to continue";  
  
    public PhoneNotVerifiedException() {  
        super(HttpStatus.FORBIDDEN, MESSAGE);  
    }  
}
```

Create **springapp/src/main/java/com/examly/springapp/sms/SmsSender.java**

```
package com.examly.springapp.sms;  
  
/** Sends one text message. Implementations: ConsoleSmsSender (development) and TwilioSmsSender. */  
public interface SmsSender {  
    /** @param toE164 phone number with country code, for example +919876543210 */  
    void send(String toE164, String message);  
}
```

Create **springapp/src/main/java/com/examly/springapp/sms/ConsoleSmsSender.java**

```
package com.examly.springapp.sms;  
  
import org.slf4j.Logger;  
import org.slf4j.LoggerFactory;  
import org.springframework.boot.autoconfigure.condition.ConditionalOnProperty;  
import org.springframework.stereotype.Component;  
  
/**  
 * Development sender: no SMS leaves the machine, the message is printed in the backend console instead.  
 * Active when otp.provider is "dev" (the default).  
 */  
@Component  
@ConditionalOnProperty(name = "otp.provider", havingValue = "dev", matchIfMissing = true)  
public class ConsoleSmsSender implements SmsSender {  
    private static final Logger log = LoggerFactory.getLogger(ConsoleSmsSender.class);  
  
    @Override  
    public void send(String toE164, String message) {  
        String masked = toE164.length() > 4 ? "*****" + toE164.substring(toE164.length() - 4) : "*****";  
        log.info("DEV SMS (not really sent) to {}: {}", masked, message);  
    }  
}
```

Create **springapp/src/main/java/com/examly/springapp/sms/TwilioSmsSender.java**

```
package com.examly.springapp.sms;  
  
import com.examly.springapp.exception.ApiCommunicationException;
```

```

import java.net.URI;
import java.net.URLEncoder;
import java.net.http.HttpClient;
import java.net.http.HttpRequest;
import java.net.http.HttpResponse;
import java.nio.charset.StandardCharsets;
import java.time.Duration;
import java.util.Base64;
import org.slf4j.Logger;
import org.slf4j.LoggerFactory;
import org.springframework.beans.factory.annotation.Value;
import org.springframework.boot.autoconfigure.condition.ConditionalOnProperty;
import org.springframework.stereotype.Component;

/**
 * Sends the SMS through Twilio's REST API (Programmable Messaging). No extra library is needed: it is one
 * HTTPS form post with Basic authentication (account SID and auth token). Active when otp.provider=twilio.
 * The three Twilio values come from environment variables and must never be written in a file.
 */
@Component
@ConditionalOnProperty(name = "otp.provider", havingValue = "twilio")
public class TwilioSmsSender implements SmsSender {
    private static final Logger log = LoggerFactory.getLogger(TwilioSmsSender.class);

    private final String accountSid;
    private final String authToken;
    private final String fromNumber;
    private final String baseUrl;
    private final HttpClient http = HttpClient.newBuilder().connectTimeout(Duration.ofSeconds(10)).build();

    public TwilioSmsSender(@Value("${twilio.account-sid}") String accountSid,
        @Value("${twilio.auth-token}") String authToken,
        @Value("${twilio.from-number}") String fromNumber,
        @Value("${twilio.base-url:https://api.twilio.com}") String baseUrl) {
        if (accountSid.isBlank() || authToken.isBlank() || fromNumber.isBlank()) {
            throw new IllegalStateException(
                "otp.provider=twilio needs TWILIO_ACCOUNT_SID, TWILIO_AUTH_TOKEN and
TWILIO_FROM_NUMBER");
        }
        this.accountSid = accountSid;
        this.authToken = authToken;
        this.fromNumber = fromNumber;
        this.baseUrl = baseUrl.endsWith("/") ? baseUrl.substring(0, baseUrl.length() - 1) : baseUrl;
    }

    @Override
    public void send(String toE164, String message) {
        String form = "To=" + enc(toE164) + "&From=" + enc(fromNumber) + "&Body=" + enc(message);
        String basic = Base64.getEncoder().encodeToString((accountSid + ":" +
authToken).getBytes(StandardCharsets.UTF_8));
        HttpRequest req = HttpRequest.newBuilder(URI.create(baseUrl + "/2010-04-01/Accounts/" + accountSid
+ "/Messages.json"))
            .timeout(Duration.ofSeconds(15))
            .header("Content-Type", "application/x-www-form-urlencoded")
            .header("Authorization", "Basic " + basic)
            .POST(HttpRequest.BodyPublishers.ofString(form))
            .build();

        try {
            HttpResponse<String> res = http.send(req, HttpResponse.BodyHandlers.ofString());
            if (res.statusCode() / 100 != 2) {
                // the body names the reason (for example an unverified trial number); it holds no secret
                log.warn("Twilio refused the SMS: status {} body {}", res.statusCode(), res.body());
                throw new ApiCommunicationException("Twilio responded with status " + res.statusCode(),
null);
            }
        } catch (ApiCommunicationException e) {
            throw e;
        } catch (InterruptedException e) {
            Thread.currentThread().interrupt();
            throw new ApiCommunicationException("Twilio request was interrupted", e);
        } catch (Exception e) {

```

```

        throw new ApiCommunicationException("Twilio request failed", e);
    }
}

private static String enc(String s) {
    return URLEncoder.encode(s, StandardCharsets.UTF_8);
}
}

```

Create `springapp/src/main/java/com/examly/springapp/dto/OtpDtos.java`

```

package com.examly.springapp.dto;

/** Request and response shapes of the OTP endpoints. */
public final class OtpDtos {
    private OtpDtos() {}

    public record OtpSendRequest(String email) {}

    public record OtpVerifyRequest(String email, String code) {}

    /**
     * required = false means the platform does not ask for OTP (nothing was sent).
     * devCode is filled only in development mode with otp.dev-expose-code=true.
     */
    public record OtpSendResponse(boolean required, String message, int resendInSeconds, String devCode) {}

    public record OtpVerifyResponse(boolean verified, String message) {}
}

```

Create `springapp/src/main/java/com/examly/springapp/service/OtpService.java`

```

package com.examly.springapp.service;

import com.examly.springapp.config.RateLimiter;
import com.examly.springapp.dto.OtpDtos.*;
import com.examly.springapp.exception.DatabaseOperationException;
import com.examly.springapp.exception.InvalidRequestException;
import com.examly.springapp.exception.PhoneNotVerifiedException;
import com.examly.springapp.exception.RateLimitExceededException;
import com.examly.springapp.model.PhoneVerification;
import com.examly.springapp.model.User;
import com.examly.springapp.repository.PhoneVerificationRepo;
import com.examly.springapp.repository.UserRepo;
import com.examly.springapp.sms.SmsSender;
import java.nio.charset.StandardCharsets;
import java.security.MessageDigest;
import java.security.SecureRandom;
import java.util.Base64;
import java.util.Locale;
import java.util.function.LongSupplier;
import org.slf4j.Logger;
import org.slf4j.LoggerFactory;
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.beans.factory.annotation.Value;
import org.springframework.core.env.Environment;
import org.springframework.core.env.Profiles;
import org.springframework.stereotype.Service;

/**
 * Mobile number verification with a one-time password (OTP).
 * <ul>
 * <li>A 6 digit code is made with SecureRandom, stored only as a salted SHA-256 hash, valid 5 minutes.
</li>
 * <li>5 wrong tries kill the code; a new code can be requested after 30 seconds, at most 5 per hour.</li>
 * <li>Answers never reveal whether an email exists or is waiting for verification.</li>
 * <li>When otp.required=false (the default) nothing is enforced, so existing flows and tests are

```

```

unchanged.</li>
* </ul>
*/
@Service
public class OtpService {
    private static final Logger log = LoggerFactory.getLogger(OtpService.class);
    static final long CODE_TTL_MS = 5 * 60_000L;
    static final int MAX_ATTEMPTS = 5;
    static final int RESEND_SECONDS = 30;
    private static final long HOUR_MS = 60 * 60_000L;
    static final String GENERIC_SEND = "If this account is waiting for verification, a code has been sent
to its mobile number.";

    private final PhoneVerificationRepo repo;
    private final UserRepo userRepo;
    private final SmsSender sms;
    private final RateLimiter limiter;
    private final boolean required;
    private final boolean exposeDevCode;
    private final String countryCode;
    private final LongSupplier clock;
    private final SecureRandom random = new SecureRandom();

    @Autowired
    public OtpService(PhoneVerificationRepo repo, UserRepo userRepo, SmsSender sms, RateLimiter limiter,
Environment env,
        @Value("${otp.required:false}") boolean required,
        @Value("${otp.dev-expose-code:false}") boolean exposeDevCode,
        @Value("${otp.country-code:+91}") String countryCode) {
        this(repo, userRepo, sms, limiter, required, exposeDevCode, countryCode,
System.currentTimeMillis);
        if (exposeDevCode && env.acceptsProfiles(Profiles.of("prod"))) {
            throw new IllegalStateException("Refusing to start: otp.dev-expose-code must be false in the
prod profile");
        }
    }

    OtpService(PhoneVerificationRepo repo, UserRepo userRepo, SmsSender sms, RateLimiter limiter, boolean
required,
        boolean exposeDevCode, String countryCode, LongSupplier clock) {
        this.repo = repo;
        this.userRepo = userRepo;
        this.sms = sms;
        this.limiter = limiter;
        this.required = required;
        this.exposeDevCode = exposeDevCode;
        this.countryCode = countryCode;
        this.clock = clock;
    }

    public boolean isRequired() {
        return required;
    }

    // ----- hooks used by registration and
login

    /** Called right after a customer or instructor registers: from now on the account must verify its
number. */
    public void markPending(User user) {
        if (!required || user == null || user.getUserId() == null) return;
        PhoneVerification row = DatabaseOperationException.guard("loading phone verification",
            () -> repo.findByUserId(user.getUserId()).orElseGet(() -> new
PhoneVerification(user.getUserId(), false));
        row.setVerified(false);
        DatabaseOperationException.guardVoid("saving phone verification", () -> repo.save(row));
    }

    /** Called by login after the password was accepted: blocks accounts that still wait for verification.
*/
    public void requireVerified(User user) {

```

```

        if (!required || user == null) return;
        if (isPending(user.getUserId())) throw new PhoneNotVerifiedException();
    }

    /** True only when a row exists and says "not verified yet". */
    public boolean isPending(Long userId) {
        return DatabaseOperationException.guard("checking phone verification",
            () -> repo.findByUserId(userId)).map(r -> !r.isVerified()).orElse(false);
    }

    // ----- send

    public OtpSendResponse send(String emailRaw, String clientId) {
        if (!required) {
            return new OtpSendResponse(false, "Mobile verification is not switched on.", 0, null);
        }
        String email = emailRaw == null ? "" : emailRaw.trim().toLowerCase(Locale.ROOT);
        if (email.isEmpty() || email.length() > 120) {
            throw new InvalidRequestException("Email is required");
        }
        // limits apply to every email, existing or not, so the answers do not reveal which emails exist
        guardRate("otp:ip:" + clientId, 20, HOUR_MS, "Too many code requests from this device. Try again
later.");
        guardRate("otp:gap:" + email, 1, RESEND_SECONDS * 1000L, "Please wait before asking for another
code.");
        guardRate("otp:hour:" + email, 5, HOUR_MS, "Too many codes requested for this account. Try again in
an hour.");

        User user = DatabaseOperationException.guard("loading a user", () ->
userRepo.findByEmail(email)).orElse(null);
        PhoneVerification row = user == null ? null
            : DatabaseOperationException.guard("loading phone verification", () ->
repo.findByUserId(user.getUserId())).orElse(null);
        String devCode = null;
        if (user != null && row != null && !row.isVerified()) {
            String code = newCode();
            String salt = newSalt();
            row.setSalt(salt);
            row.setCodeHash(hash(salt, code));
            row.setAttempts(0);
            row.setLastSentAtMs(clock.getAsLong());
            row.setExpiresAtMs(clock.getAsLong() + CODE_TTL_MS);
            DatabaseOperationException.guardVoid("saving the code", () -> repo.save(row));
            sms.send(toE164(user.getMobileNumber()),
                "Your E-Learning Platform verification code is " + code + ". It expires in 5 minutes.
Do not share it.");
            log.info("OTP sent for user {}", user.getUserId());
            if (exposeDevCode) devCode = code;
        }
        return new OtpSendResponse(true, GENERIC_SEND, RESEND_SECONDS, devCode);
    }

    // ----- verify

    public OtpVerifyResponse verify(String emailRaw, String code, String clientId) {
        if (!required) {
            return new OtpVerifyResponse(true, "Mobile verification is not switched on.");
        }
        String email = emailRaw == null ? "" : emailRaw.trim().toLowerCase(Locale.ROOT);
        if (email.isEmpty() || code == null || !code.matches("[0-9]{6}$")) {
            throw new InvalidRequestException("Enter the 6 digit code");
        }
        guardRate("otp:verify-ip:" + clientId, 30, HOUR_MS, "Too many attempts from this device. Try again
later.");

        User user = DatabaseOperationException.guard("loading a user", () ->
userRepo.findByEmail(email)).orElse(null);
        PhoneVerification row = user == null ? null
            : DatabaseOperationException.guard("loading phone verification", () ->
repo.findByUserId(user.getUserId())).orElse(null);
        if (row == null || row.isVerified() || row.getCodeHash() == null

```

```

        || clock.getAsLong() > row.getExpiresAtMs() || row.getAttempts() >= MAX_ATTEMPTS) {
            throw new InvalidRequestException("This code is wrong or has expired. Request a new code.");
        }
        boolean match = MessageDigest.isEqual(
            hash(row.getSalt(), code).getBytes(StandardCharsets.UTF_8),
            row.getCodeHash().getBytes(StandardCharsets.UTF_8));
        if (!match) {
            row.setAttempts(row.getAttempts() + 1);
            DatabaseOperationException.guardVoid("saving the attempt", () -> repo.save(row));
            int left = MAX_ATTEMPTS - row.getAttempts();
            throw new InvalidRequestException(left > 0
                ? "Wrong code. " + left + (left == 1 ? " try" : " tries") + " left."
                : "Too many wrong codes. Request a new code.");
        }
        row.setVerified(true);
        row.setCodeHash(null);
        row.setSalt(null);
        row.setAttempts(0);
        row.setVerifiedAtMs(clock.getAsLong());
        DatabaseOperationException.guardVoid("saving verification", () -> repo.save(row));
        log.info("Mobile number verified for user {}", user.getUserId());
        return new OtpVerifyResponse(true, "Your mobile number is verified. You can log in now.");
    }

    // ----- helpers

    private void guardRate(String key, int limit, long windowMs, String message) {
        if (!limiter.tryAcquire(key, limit, windowMs)) {
            throw new RateLimitExceededException(message, limiter.retryAfterSeconds(key, windowMs));
        }
    }

    String toE164(String mobile) {
        String digits = mobile == null ? "" : mobile.replaceAll("[^0-9]", "");
        return countryCode + digits;
    }

    private String newCode() {
        return String.valueOf(100000 + random.nextInt(900000));
    }

    private String newSalt() {
        byte[] b = new byte[12];
        random.nextBytes(b);
        return Base64.getUrlEncoder().withoutPadding().encodeToString(b);
    }

    static String hash(String salt, String code) {
        try {
            byte[] d = MessageDigest.getInstance("SHA-256").digest((salt + ":" +
            code).getBytes(StandardCharsets.UTF_8));
            return Base64.getUrlEncoder().withoutPadding().encodeToString(d);
        } catch (java.security.NoSuchAlgorithmException e) {
            throw new IllegalStateException(e);
        }
    }
}

```

Create `springapp/src/main/java/com/examly/springapp/controller/OtpController.java`

```

package com.examly.springapp.controller;

import com.examly.springapp.dto.OtpDtos.*;
import com.examly.springapp.service.OtpService;
import io.swagger.v3.oas.annotations.Operation;
import io.swagger.v3.oas.annotations.tags.Tag;
import jakarta.servlet.http.HttpServletRequest;
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.http.ResponseEntity;

```



```
import org.springframework.web.bind.annotation.*;

/** Mobile number verification by OTP. Public (the user cannot log in yet); abuse is limited inside
OtpService. */
@RestController
@RequestMapping("/api/otp")
@Tag(name = "Mobile OTP", description = "Send and check the verification code")
public class OtpController {
    private final OtpService otp;

    @Autowired
    public OtpController(OtpService otp) {
        this.otp = otp;
    }

    @Operation(summary = "Send a verification code",
        description = "Body: {email}. The code goes to the mobile number saved at registration. The
answer is the same for every email.")
    @PostMapping("/send")
    public ResponseEntity<OtpSendResponse> send(@RequestBody OtpSendRequest body, HttpServletRequest
request) {
        return ResponseEntity.ok(otp.send(body == null ? null : body.email(), request.getRemoteAddr()));
    }

    @Operation(summary = "Check the verification code", description = "Body: {email, code}. 6 digits, 5
tries, valid 5 minutes.")
    @PostMapping("/verify")
    public ResponseEntity<OtpVerifyResponse> verify(@RequestBody OtpVerifyRequest body, HttpServletRequest
request) {
        return ResponseEntity.ok(otp.verify(body == null ? null : body.email(), body == null ? null :
body.code(), request.getRemoteAddr()));
    }

    @Operation(summary = "Is OTP switched on?")
    @GetMapping("/status")
    public ResponseEntity<java.util.Map<String, Boolean>> status() {
        return ResponseEntity.ok(java.util.Map.of("required", otp.isRequired()));
    }
}

```

Step 2: three small edits in files you already have

2a. `controller/AuthController.java` (3 edits inside this one file)

Edit 1: add one import with the other imports at the top (for example under `import` `com.examly.springapp.service.RefreshTokenService;`):

```
import com.examly.springapp.service.OtpService;
```

Edit 2: add one field under the line `private final long refreshDays;` (the other fields):

```
@Autowired(required = false)
private OtpService otpService;
```

Edit 3: in `register(...)`, find this line:

```
User saved = userService.register(user);
```

and put this directly **under** it:

```
if (otpService != null) {
    otpService.markPending(saved);
}
```

Edit 4: in `login(...)`, find this line:

```
loginAttempts.recordSuccess(key);
```

and put this directly **under** it:

```
if (otpService != null) {
    otpService.requireVerified(user); // throws 403 when the mobile number is not verified yet
}
```

(That makes four small pastes in one file: `import`, `field`, `register`, `login`.)

2b. `config/SecurityConfig.java`

The OTP endpoints must be reachable **without** logging in (the person cannot log in yet). Find the rule that lists the public paths:

```
.requestMatchers("/api/register", "/api/login", "/api/refresh", "/api/logout", "/api/health")
```

and add `"/api/otp/**"` to that list:

```
.requestMatchers("/api/register", "/api/login", "/api/refresh", "/api/logout", "/api/health",
"/api/otp/**")
```

(The line below it already says `.permitAll()`. Spring uses the **first** matching rule, so this must stay above `.anyRequest().authenticated()`.)

2c. `src/main/resources/application.properties`

Add these lines at the very **end** of the file. For this test they are written with the values you need **directly** (no environment variables to set):

```
# ---- Mobile OTP (Day 1) ----
otp.required=true
otp.provider=dev
otp.dev-expose-code=true
otp.country-code=+91
twilio.account-sid=
twilio.auth-token=
twilio.from-number=
```

What they mean: `required=true` turns the OTP rule on. `provider=dev` prints the code in the console instead of sending an SMS. `dev-expose-code=true` also returns the code in the Swagger answer so you can copy it. **Before a real demo or deployment set `otp.dev-expose-code=false`** (the app refuses to start with it `true` in the `prod` profile).

If you prefer environment variables, use `otp.required=${OTP_REQUIRED:false}` style lines from the long guide; the dev values above are simply the quickest way to test.

Step 3: start the backend

1. Start MySQL.
2. Run the backend as you always do (`mvn spring-boot:run` or the IntelliJ Run button).
3. It must start with **no red error**. If you see one, copy the first red lines and send them to me.
4. Check the new table was created. In MySQL:

```
SELECT * FROM phone_verifications;
```

It exists and is empty (an error "table doesn't exist" means the app did not start properly or `ddl-auto` is not `update`).

Step 4: test everything in Swagger (about 5 minutes)

Open <http://localhost:8080/swagger-ui.html>. For each call: click the endpoint, click **Try it out**, paste the JSON into the body box, click **Execute**, and compare with "You should see".

Use an email that has **never** been registered. If you run the test twice, change the email (for example `asha2@mail.com`).

Call 1: is OTP on? `GET /api/otp/status` (group "Mobile OTP")

No body. **You should see:** status `200`, body `{"required": true}`.

Call 2: register `POST /api/register`

```
{
  "username": "asha",
  "email": "asha@mail.com",
  "password": "Secret@123",
  "mobileNumber": "9876543210",
  "role": "CUSTOMER"
}
```

You should see: status `201`. (The mobile number must be 10 digits starting with 6, 7, 8 or 9.) Now check the database: `SELECT * FROM phone_verifications;` shows **one row** with `verified = 0` (false).

Call 3: try to log in before verifying `POST /api/login`

```
{
  "email": "asha@mail.com",
  "password": "Secret@123"
}
```

You should see: status `403` and a message **"Please verify your mobile number to continue"**. This is the proof that the lock works.

Call 4: ask for the code `POST /api/otp/send`

```
{
  "email": "asha@mail.com"
}
```

You should see: status `200` and a body like:

```
{
  "required": true,
  "message": "If this account is waiting for verification, a code has been sent to its mobile number.",
  "resendInSeconds": 30,
  "devCode": "483920"
}
```

Copy the `devCode` (your number will differ). The same code is printed in the backend console on a line like `DEV SMS (not really sent) to *****3210: Your E-Learning Platform verification code is 483920 ...`.

Call 5: type a wrong code on purpose `POST /api/otp/verify`

```
{
  "email": "asha@mail.com",
  "code": "000000"
}
```

You should see: status `400`, message **"Wrong code. 4 tries left."** (After 5 wrong tries the code dies and you must ask for a new one.)

Call 6: the right code `POST /api/otp/verify`

```
{
  "email": "asha@mail.com",
  "code": "PASTE-THE-devCode-HERE"
}
```

You should see: status `200`, `{"verified": true, "message": "Your mobile number is verified. You can log in now."}`. The database row now says `verified = 1` and the hash is cleared.

Call 7: log in again `POST /api/login`

Same body as Call 3. You should see: status `200` with a `token`. The lock is gone.

Extra checks (optional, 2 minutes)

Try	You should see
<code>POST /api/otp/send</code> twice within 30 seconds	the second one answers 429 "Please wait before asking for another code."
<code>POST /api/otp/send</code> with an email that does not exist	200 with the same message (on purpose: it never reveals which emails are registered) and no <code>devCode</code>
Wait 5 minutes after sending, then verify	400 "This code is wrong or has expired. Request a new code."

Try	You should see
Log in as your old admin / any account created before today	works normally (they have no row, so they count as verified)
Set <code>otp.required=false</code> and restart	GET <code>/api/otp/status</code> says <code>false</code> , and register/login behave exactly as before

If something goes wrong

What you see	Reason	Fix
Red error at start: cannot find symbol ... <code>OtpService</code> in <code>AuthController</code>	the import (Edit 1) is missing	add the import
Red error: <code>package com.examly.springapp.sms</code> does not exist	the <code>sms</code> folder or its files are in the wrong place	create <code>sms</code> under <code>com/examly/springapp/</code>
Red error: <code>RateLimiter</code> not found	CHANGES-7 was not applied	apply CHANGES-7 first
Start error: "Refusing to start: <code>otp.dev-expose-code</code> "	you run the <code>prod</code> profile	set <code>otp.dev-expose-code=false</code>
Call 3 gives 200 (login works before verifying)	<code>otp.required</code> is not <code>true</code> , or Edit 4 is missing, or the account existed before OTP was on	check the properties line and Edit 4; use a new email
Call 4 answers 200 but no <code>devCode</code>	<code>otp.dev-expose-code</code> is not <code>true</code> , or the account is not waiting (already verified, or registered before <code>required=true</code>)	check the properties; register a new email
Call 4: 401 or 403	the <code>/api/otp/**</code> rule is missing or below a stricter rule	redo Step 2b
Call 4: 404	<code>OtpController</code> is missing or in the wrong package	recheck Step 1
<code>phone_verifications</code> has no row after registering	Edit 3 is missing, or <code>otp.required</code> is false	check both
Call 4: 429	you asked again within 30 seconds, or more than 5 times an hour	wait; or use a new email

Later: real SMS with Twilio

Not needed for the demo. When you want it: `DAY1-OTP-BACKEND.pdf` section 6 explains Twilio step by step. The only change is three environment variables and `otp.provider=twilio`; none of the code above changes.

Honest status

The OTP rules were compiled and run as real Java against fake repositories: **33 checks passed, 0 failed** (hashing, expiry, 5-try lock, 30-second gap, hourly limit, same answer for unknown emails, old users counted as verified). The register-then-verify screens were tested in a real browser against a pretend backend. **Not tested:** this exact Spring start-up with your real MySQL (Maven is blocked in my workspace). If the backend shows a red error, send me the first red lines and I will fix it.